Since we changed the entity model, we need to update the database schema accordingly. Let's assume that Catalog Service is already in production, so we need a Flyway migration to update the schema in the next release. In chapter 5, we introduced Flyway to add version control to our databases. Each change to a schema must be registered as a migration, ensuring robust schema evolution and reproducibility.

Any change to a database schema should also be backward compatible to support common deployment strategies for cloud native applications, like rolling upgrades, blue/green deployments, or canary releases (a subject we'll cover in chapter 15). In this case, we need to add new columns to the `book` table. As long as we don't make them mandatory, the change will be backward compatible. After we change the schema, any running instance of the previous release of Catalog Service will continue to work without errors, simply ignoring the new columns.

In the src/main/resources/db/migration folder for the Catalog Service project, create a new `V3__Add_user_audit.sql` migration script to add two new columns to the `book` table. Ensure you type two underscores after the version number.

Listing 12.24 Adding new audit metadata to the `book` table

```
ALTER TABLE book
  ADD COLUMN created_by varchar(255);          ❶
ALTER TABLE book
  ADD COLUMN last_modified_by varchar(255);    ❷
```

❶ Add a column to hold the username of who created the row.

❷ Add a column to hold the username of who last updated the row.

During the application's startup, Flyway will automatically go through all the migration scripts and apply those not yet applied.